생성형 AI 활용

Dept. of Data Science, Kangwon National Univ.
Lee, Won Sang, *Ph.D*
Associate Professor
Director, Gangneung Startup Support Center
CEO, Unique AI

Table of Contents

I. AI와 LLM 이해하기

II. LLM 활용과 프롬프팅

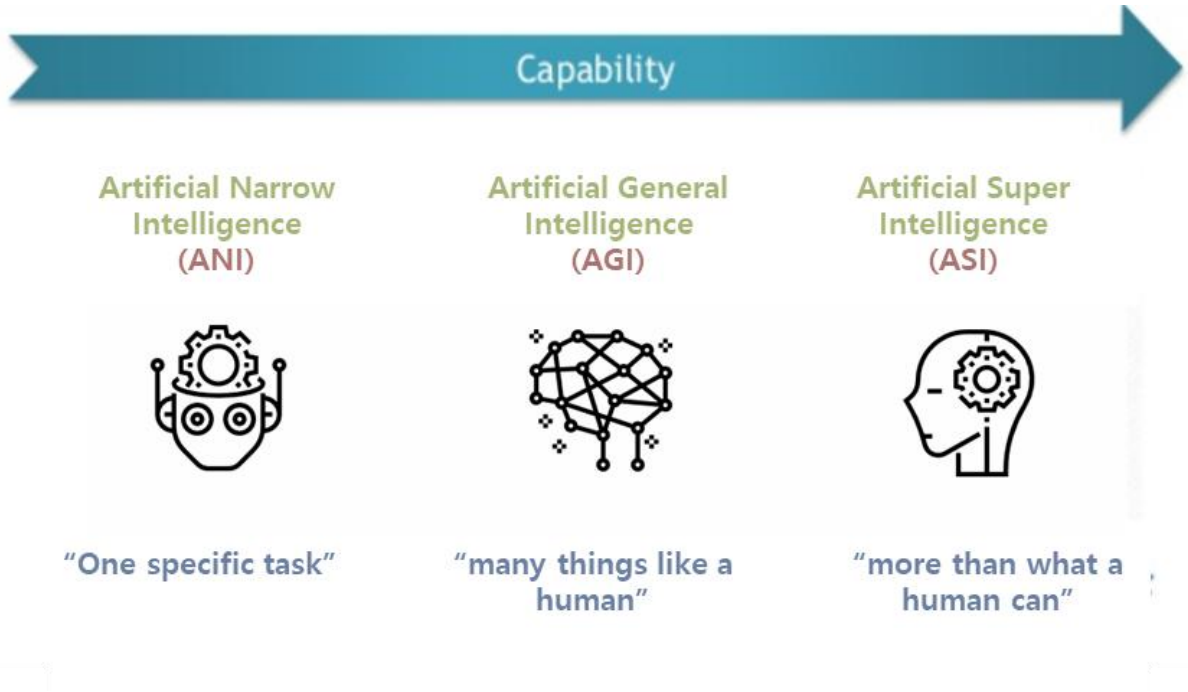
III. Work, Rag, Finetuning

IV. Vibe coding 소개

V. MCP 소개

VI. LLM 실습

- **AI 개요**



- '약인공지능(ANI)', '범용인공지능(AGI)', '초인공지능(ASI)'
- ANI: 주어진 데이터 내에서 주어진 문제를 해결하는 AI
- AGI로의 진입 가속화 중

- **생성형 AI**
  - 대형 언어모델이나 이미지 생성모델 활용, User가 원하는 것을 생성하는 AI 분야
  - LLM: 테라바이트 단위의 대용량 코퍼스로 모형을 학습, 문장을 생성하는 모형



※ 자료: 삼일PwC경영연구원

- **Scaling Laws (Open AI, 2020)**
  - 컴퓨팅 리소스, 데이터, 모형 크기를 늘릴 수록 성능 개선
  - 새로운 능력이 생겨남 (Emergent Abilities)

Scaling Laws for Neural Language Models

Jared Kaplan *
Johns Hopkins University, OpenAI
jaredk@jhu.edu

Sam McCandlish*
OpenAI
sam@openai.com

Tom Henighan
OpenAI
henighan@openai.com

Tom B. Brown
OpenAI
tom@openai.com

Benjamin Chess
OpenAI
bchess@openai.com

Rewon Child
OpenAI
rewon@openai.com

Scott Gray
OpenAI
scott@openai.com

Alec Radford
OpenAI
alec@openai.com

Jeffrey Wu
OpenAI
jeffwu@openai.com

Dario Amodei
OpenAI
damodei@openai.com

Abstract

We study empirical scaling laws for language model performance on the cross-entropy loss. The loss scales as a power-law with model size, dataset size, and the amount of compute used for training, with some trends spanning more than seven orders of magnitude. Other architectural details such as network width or depth have minimal effects within a wide range. Simple equations govern the dependence of overfitting on model/dataset size and the dependence of training speed on model size. These relationships allow us to determine the optimal allocation of a fixed compute budget. Larger models are significantly more sample-efficient, such that optimally compute-efficient training involves training very large models on a relatively modest amount of data and stopping significantly before convergence.

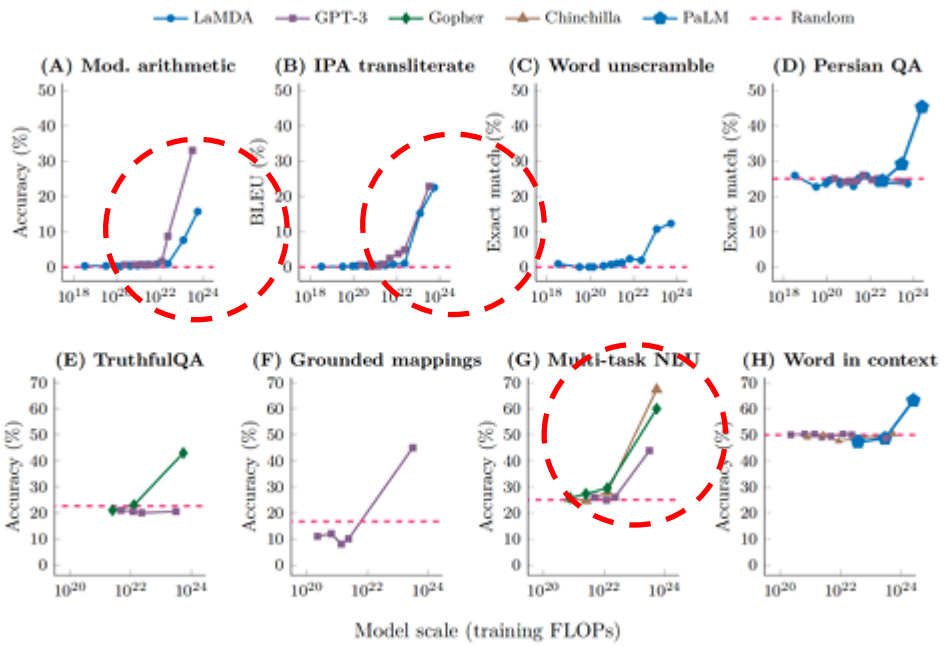
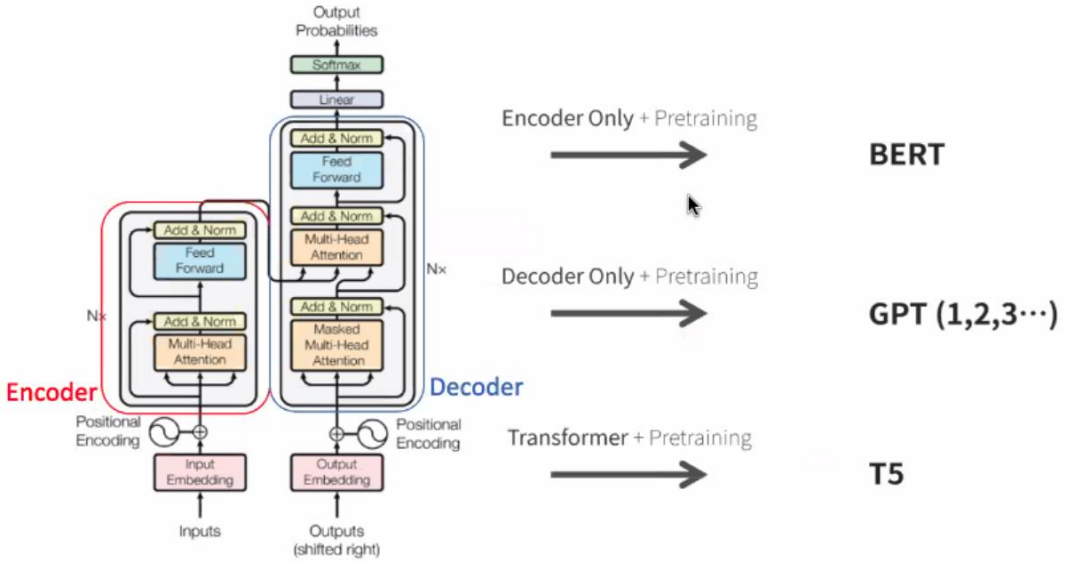


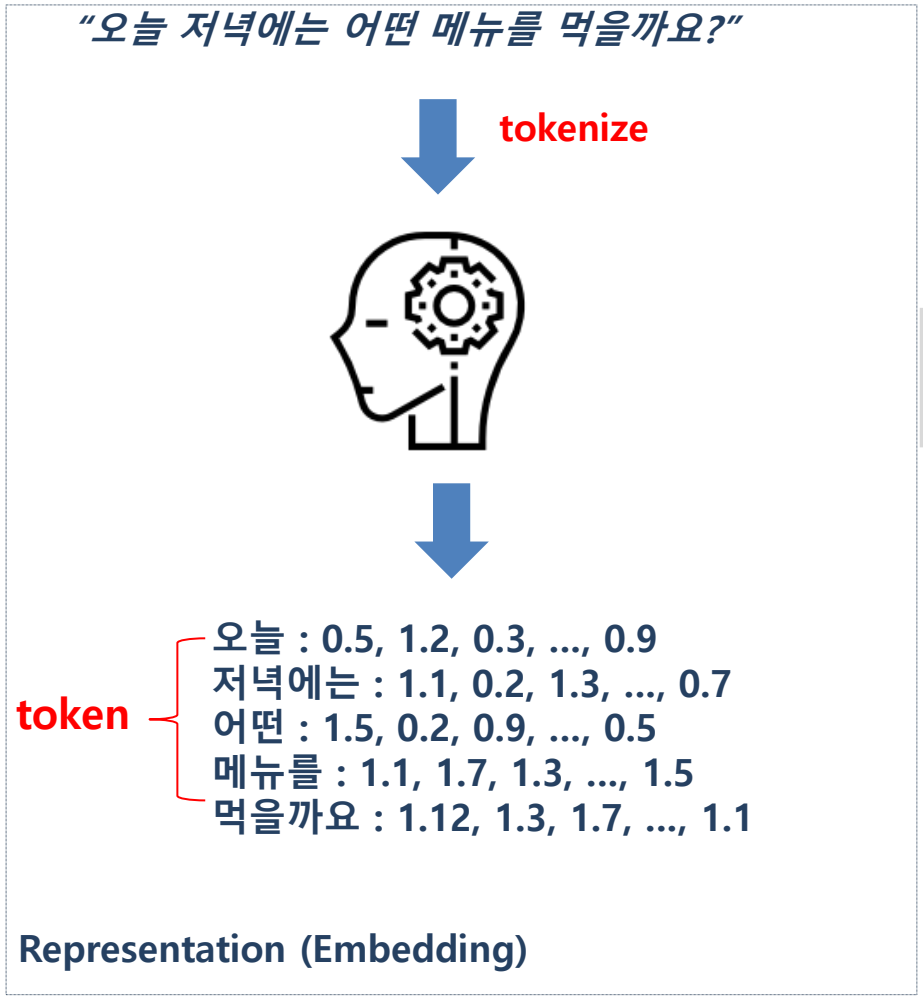
Figure 1: **Emergent abilities of large language models**. Model families display *sharp* and *unpredictable* increases in performance at specific tasks as scale increases. Source: Fig. 2 from [33].

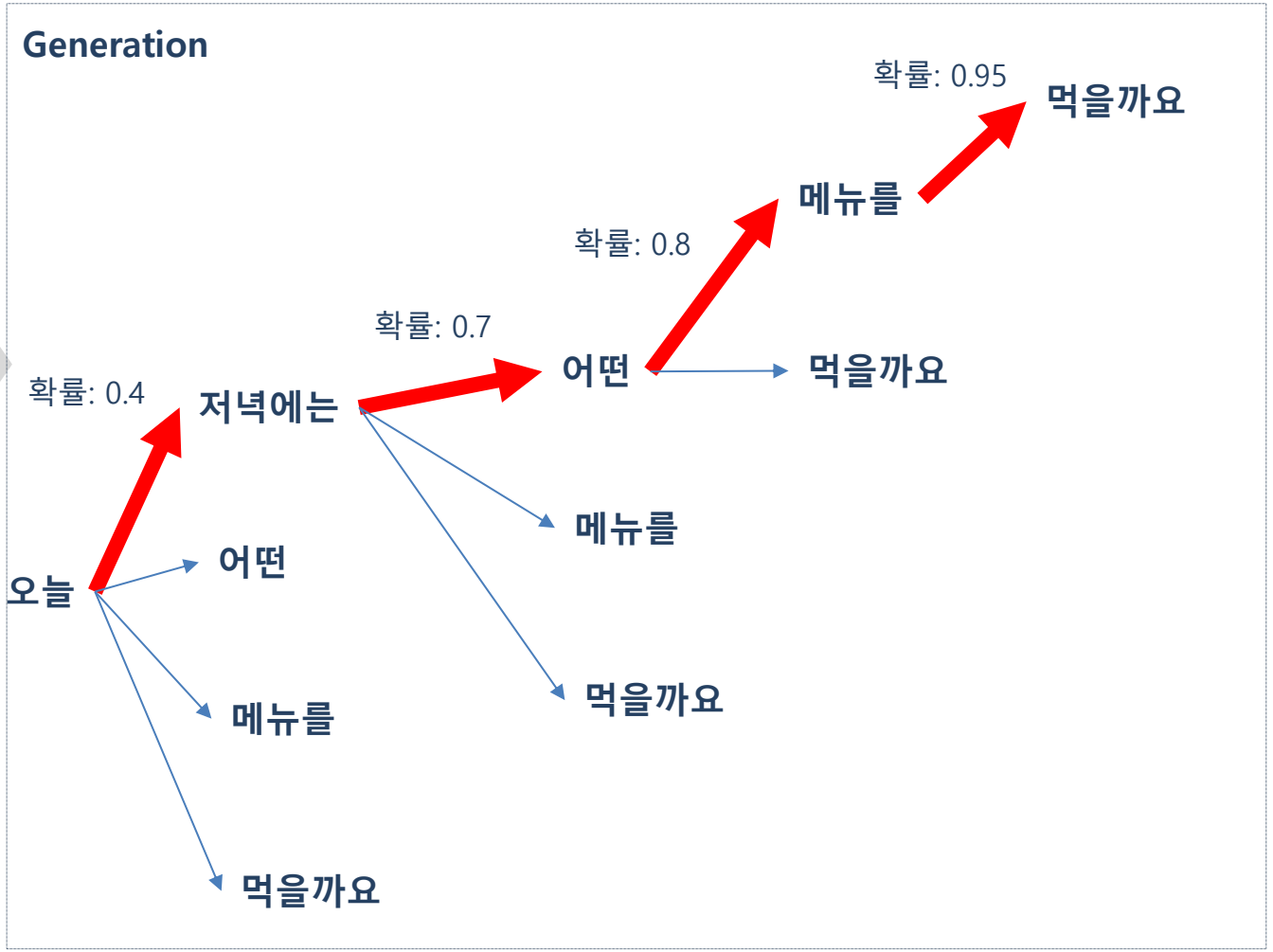
- **간단하게 이해하는 LLM**
  - 언어모형: 주어진 문장을 구성하는 단어에 대한 확률을 모델링
  - 주어진 문장/단어의 다음 단어를 예측, 문장 내 Masking한 단어를 예측
  - 대용량 텍스트를 학습한 모형이 좋은 Representation(Embedding)을 보유



I. AI와 LLM 이해하기

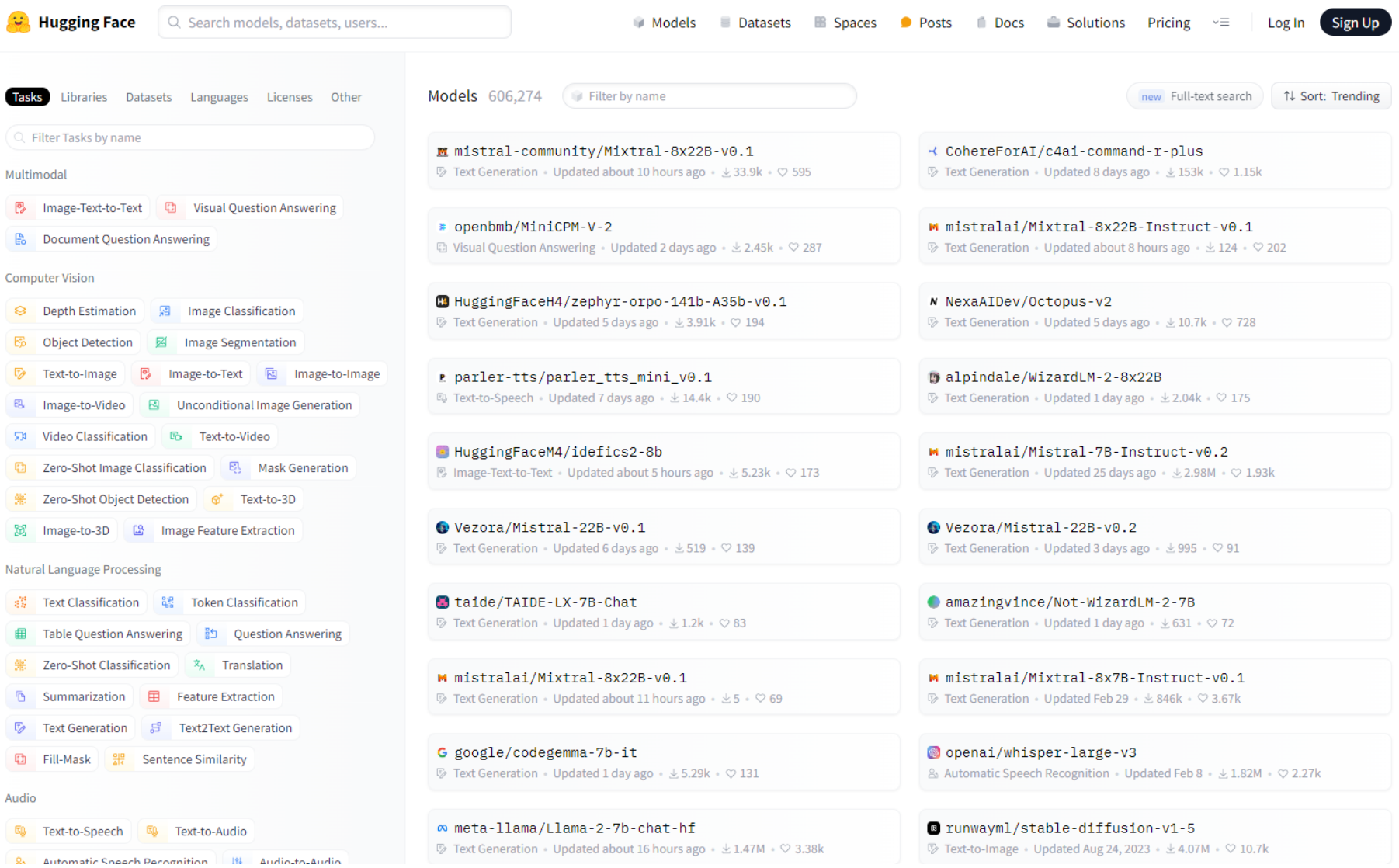
- 간단하게 이해하는 LLM





I. AI와 LLM 이해하기

- 다양한
- LLM



- **챗GPT**
  - GPT3는 GPT2에 비해 10배 이상의 데이터(570GB), 100배 이상 모형 크기(175B)
  - GPT4는 1700B의 파라미터(estimated)
  - GPT5?

  - Few-shot 학습을 통한 In-Context Learning 향상
    - 프롬프트의 내용만으로 Task 수행
    - 프롬프트 내 맥락을 이해하고 답변

II. LLM 활용과 프롬프팅

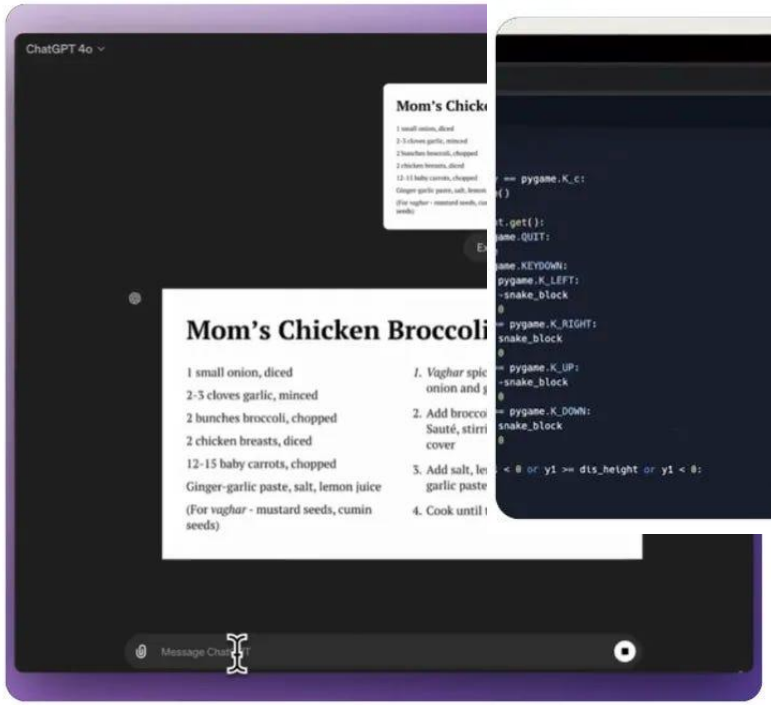
- **프롬프팅을 통한 업무활용: 이미지에서 텍스트 추출, 데이터 분석, 코딩 등...**

Robin Delta
@heyrobinai

3. Extract text from images

Prompt: Extract this recipe to a JSON file

Traducir post



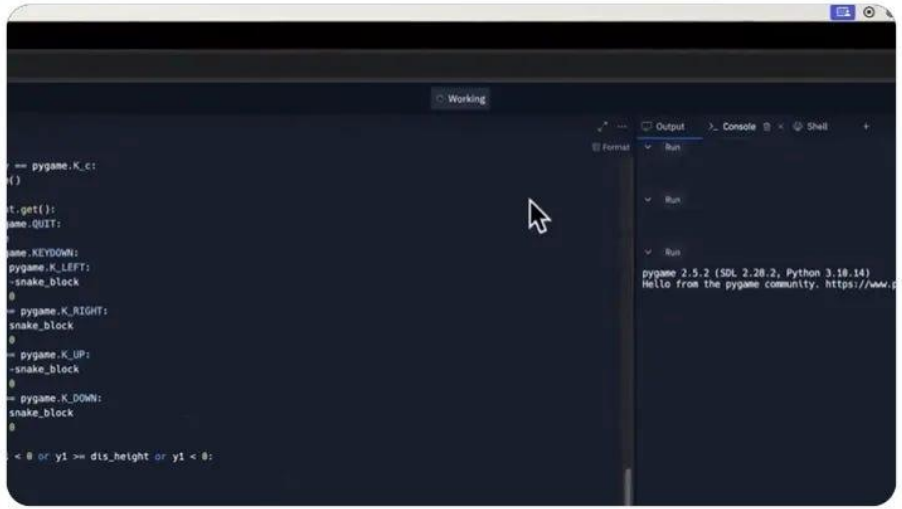
Rowan Cheung
@rowancheung

The ChatGPT Mac app is the ultimate screenshot-to-code tool.

Screenshot anything, paste it in the ChatGPT shortcut, and just tell GPT-4o to code it for you.

Here's me taking a snapshot of Snake Game and getting fully working code in 90 seconds. Video is on 3x speed.

Traducir post

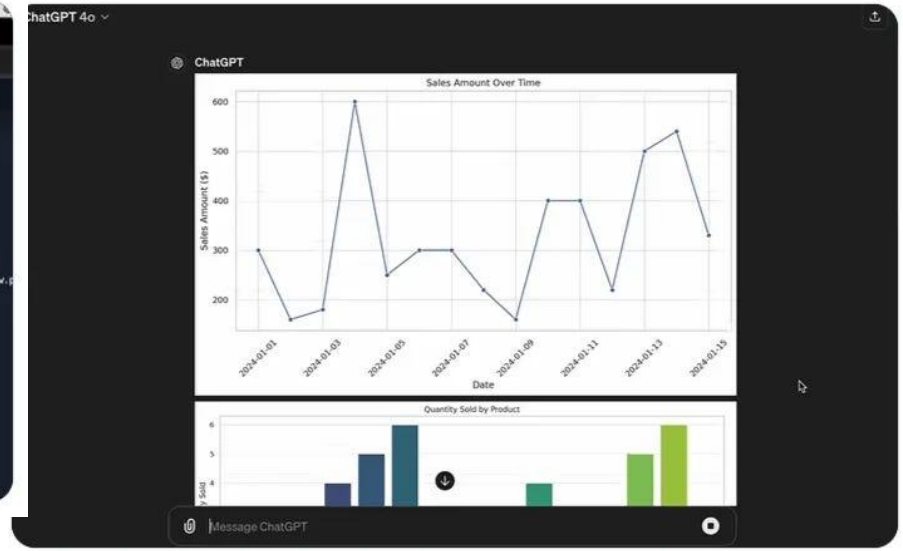


Zain Kahn
@heykahn

/ild. GPT-4o on ChatGPT can generate full blown charts and statistical nalysis from spreadsheets with a single prompt in less than 30 econds.

his stuff use to take ages in Excel.

aducir post



- **프롬프트 구조화**
  - Delimiter와 프롬프트 내 영역 지정
    - Delimiter: 특정 토큰으로 프롬프트에서 영역을 지정할 때 활용
    - ###, >>>, ===, <태그> 등으로 표현, 각 영역이 의도에 맞게 챗GPT에 전달되도록 함
    - **활용 예시 프롬프트**

```
대화 예시와 결과 예시를 참고해서, 다음 <<<대화>>>에 나오는 각 문장의 감성을 '긍정 ' 이나 '부정 ' 으로 분류하시오.
###
대화 예시
[행인1]: 오늘 날씨가 좋네요
[행인2]: 좋긴 뭐가 좋아요
###
결과 예시
긍정
부정
###
<<<
[행인1] 비가 오니 차분합니다.
[행인2] 비오는 날 분위기 좋아요
>>>
```

- **프롬프트 구조화**
  - **XML 태그 스타일의 영역 지정**
    - **XML태그?**
      - XML은 데이터의 구조를 기술하는 Mark up 언어
      - 데이터의 각 항목에 대해 의미하는 태그로 마크업
      - 예: <키워드> 챗, 생성AI </키워드>

- **프롬프트 구조화**
  - **활용 예시 프롬프트(XML스타일)**

대화 예시와 결과 예시를 참고해서, 다음 <<<대화>>>에 나오는 각 문장의 감성을 '긍정 ' 이나 '부정 ' 으로 분류하시오.

<범주>
긍정
부정
</범주>
<대화예시>
[행인1]: 오늘 날씨가 좋네요
[행인2]: 좋긴 뭐가 좋아요
</대화예시>
<결과예시>
긍정
부정
</결과예시>
<대화>
[행인1] 비가 오니 짜증납니다.
[행인2] 비오는 날 분위기 좋아요
</대화>

- **프롬프트 구조화**
  - 시스템 프롬프트 = 맞춤형 지침
    - **“답변의 규칙 "**
    - 챗GPT가 어떻게 응답해야 하는지를 알려주는 지시사항 또는 가이드에 대한 프롬프트
    - 입력하려는 질문 앞에서 제시함
    - 이후 나오는 프롬프트에 대해 지속적으로 적용

    - 주요 내용
      - 태스크 정의
      - 출력 형식 정의
      - 준수사항 정의: 피해야 할 응답 등

II. LLM 활용과 프롬프팅

- **프롬프팅 관련 기능: 메모리, 아카이브, 컨텍스트, 챗**
  - 챗: 대화
  - 컨텍스트: 챗의 모음
  - 대화: 챗과 컨텍스트의 모음, 2개월간 저장
  - 아카이브: 영구 저장된 대화
  - 메모리: 아카이브 중 주요 내용을 항상 활용할 수 있도록 준비

II. LLM 활용과 프롬프팅

- **메모리**
  - 사용자와의 이전 대화를 기억하고,이후 대화에 지속적으로 반영하는 기능
  - 반복 설명 없이, 개인화된 대화 가능
  - 메모리 관리 방법
    - 좌측 메뉴 > 프로필 > "메모리" 메뉴 켜기 / 끄기
  - 개별 정보 삭제 가능 + 각 대화별로도 비활성화 가능
  - 민감한 정보(비밀번호, 개인정보 등) 는 저장하지 않음
  - 챗GPT가 기억한 내용은 수동으로 확인 및 수정 가능
    - 메모리는 대화 중 자동으로 생성되며, GPT가 "이 내용을 기억해도 될까요?"라고 확인

III. Work, Rag, Finetuning

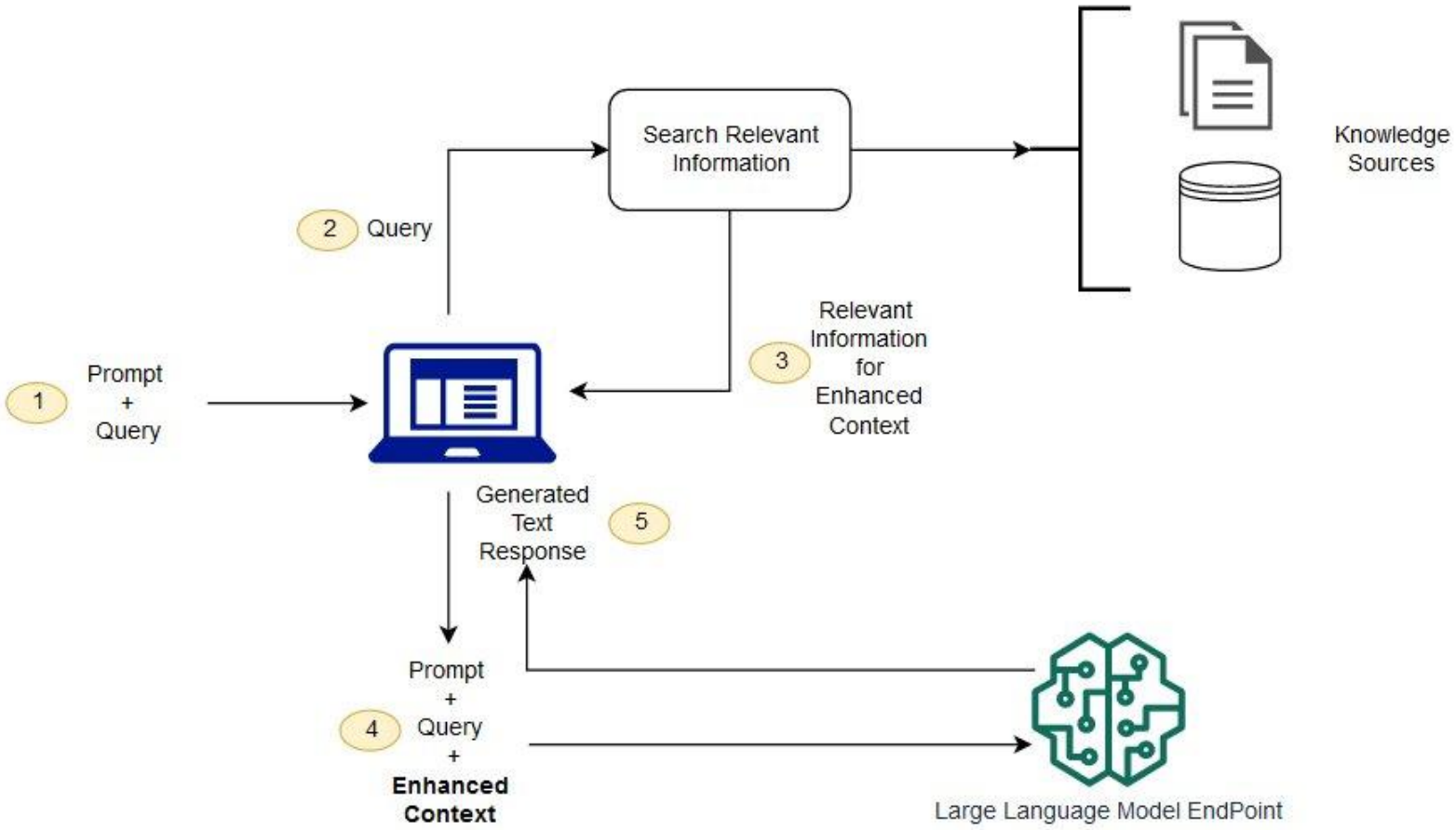
- **챗GPT Work모드: Agent, Agentic AI**
  - **복잡한 업무를 목표부터 결과물까지 수행하는 AI 에이전트**
    - 단순 질의응답이 아니라 여러 단계의 업무를 계획하고 순차적으로 수행
    - 작업 중 사용자가 방향 수정 및 주요 작업 승인 가능
      - 정보 조사 및 분석: 웹, 첨부파일, 프로젝트 자료 등을 활용하여 필요한 정보 수집, 수집된 자료를 분석·요약하여 업무 결과로 연결
      - 앱/파일 연계 작업: 연결된 앱과 파일을 넘나들며 필요한 자료를 활용, 예를 들어 Google Docs/Sheets/Slides 등과 연계 가능
      - 완성된 결과물 생성: 문서, 보고서, 스프레드시트, 프레젠테이션, 분석자료, Site 등을 직접 생성
    - *기존 파일이나 템플릿을 기반으로 수정/확장 가능*
    - *반복/지속 업무 자동화*
    - *한 번 실행하는 작업뿐 아니라 일정 기반 반복 실행*
    - *특정 변화 발생 여부 모니터링 지원*

- **RAG: 검색 증강 생성**
  - Retrieval-Augmented Generation
  - Hallucination을 줄이는 비용 효율적인 접근 방식
    - LLM의 기능을 특정 도메인이나 특정 기관의 내부 지식 기반으로 확장
    - LLM 외부의 데이터를 활용해서 보다 정확한 답변을 찾는 방안
    - 모형 학습을 다시 할 필요는 없음

III. Work, Rag, Finetuning

- **RAG 과정**

- LLM 학습 데이터 외의 데이터인 외부 데이터 생성 후 임베딩을 통해 벡터 DB저장
- 질문에 대한 관련 정보 검색
- 외부 데이터 업데이트: 실시간 및 배치 방식



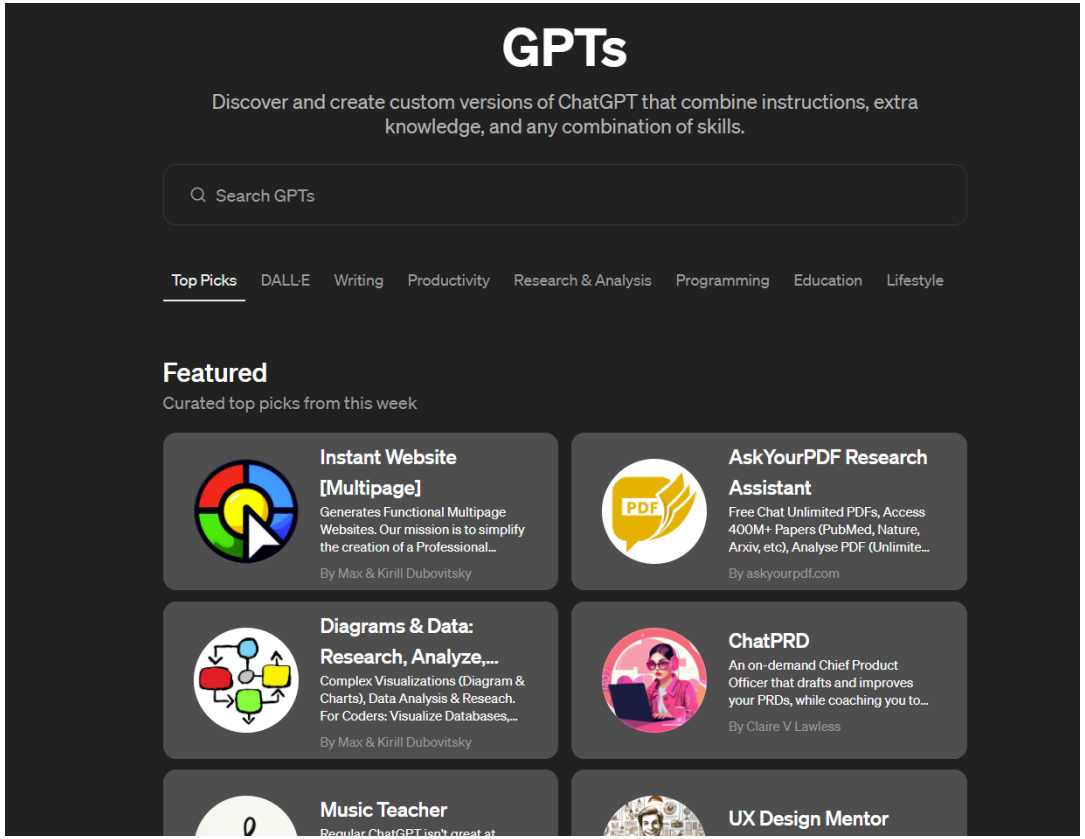
- **RAG: 검색 증강 생성**
  - 1단계:
    - 사용자 질문 입력사용자가 질문과 함께 프롬프트 입력
    - 예: "2026년 전기차 보조금 정책은 어떻게 바뀌었는지 알려줘"
  - 2단계:
    - 질문 → 웹 검색 쿼리로 변환
    - 핵심 키워드 중심의 간결한 쿼리 생성예: "2025년 한국 전기차 보조금 변경 내용"
  - 3단계:
    - 웹 검색 및 정보 수집정부/언론 등 신뢰 가능한 출처 탐색
    - 주요 문장 추출 및 요약예: "보조금 600만 원 → 500만 원 축소, 고가 차량 제외"

- **RAG: 검색 증강 생성**
  - 4단계:
    - 확장된 문맥 생성 (Enhanced Context)
    - 외부 정보 + 프롬프트 + 사용자 질문 결합→ LLM 입력용 고도화된 문맥 생성
  - 5단계:
    - LLM이 최종 응답 생성요약, 설명, 근거 등 사용자 맞춤 응답 출력
    - 예: "보조금이 500만 원으로 축소되며, 고가 차량은 제외..."

- **Fine Tuning**
  - 기존 GPT 모델에 특정 데이터셋을 추가 학습시켜 도메인 특화 모델을 만드는 방법
  - GPT의 기본 성능 + 맞춤형 지식/스타일 내재화
  - Fine Tuning의 특징
    - 일회성 응답이 아닌 지속적으로 반영되는 학습 결과
    - 다음을 모델 자체에 자동 학습
      - 도메인 지식
      - 브랜드 어조
      - 업무 스타일 등

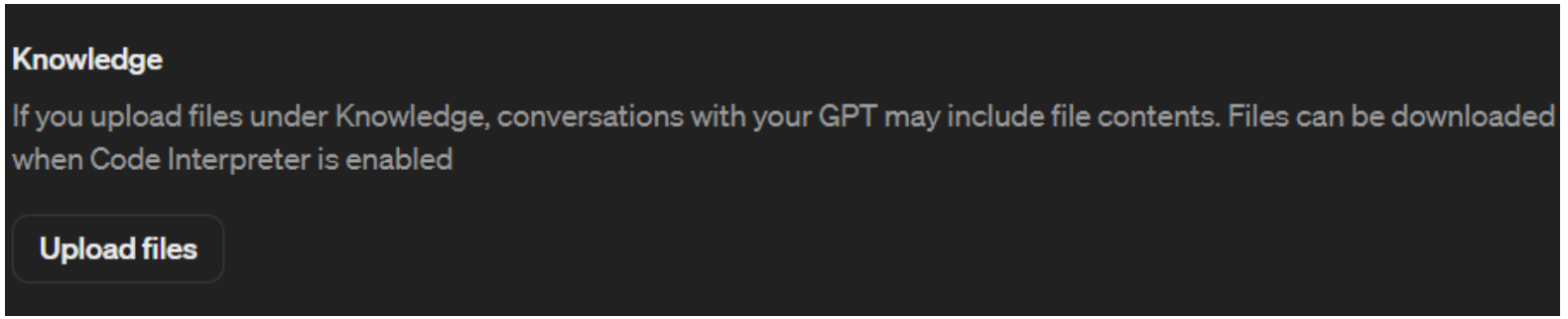
III. Work, Rag, Finetuning

- **GPT**
  - GPT: Customize된 챗GPT를 의미
    - instruction: 제약사항, 가이드라인, 설명, 개인화 등을 확인

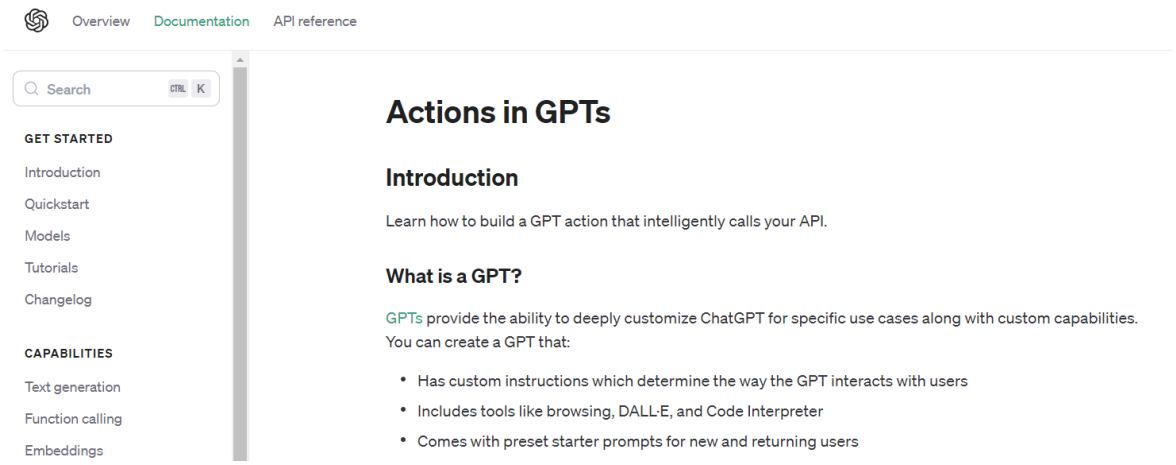


III. Work, Rag, Finetuning

- **GPT 설정**
  - Knowledge에 파일 업로드



  - 외부 API의 활용



III. Work, Rag, Finetuning

- **GPT 생성**

  - **Actions**
    - API? API는 컴퓨터나 컴퓨터 프로그램 사이의 연결이다. 일종의 소프트웨어 인터페이스이며 다른 종류의 소프트웨어에 서비스를 제공
    - YAML, JSON, 그 외 양식 지원
  - **참고:** https://platform.openai.com/docs/actions/introduction





IV. Vibe coding 소개

- **Vibe 코딩**
  - Vibe 코딩(Vibe Coding)이란?
    - AI 중심 소프트웨어 개발 방식
    - Andrej Karpathy(OpenAI 공동 창업자)가 제안
    - 기존의 명시적 코딩 → 자연어 지시 + AI 응답
  - Vibe 코딩의 핵심 철학
    - "보고 말하고 실행하고 복사하여 붙여넣는 일련의 행위"
    - 기술/언어 제약 없이, 아이디어를 결과물로 빠르게 전환



IV. Vibe coding 소개

- **Vibe 코딩**
  - 직관적 개발
    - 자연어로 명령, 자동 코드 생성
      - "캔버스에서 장애물을 피하는 게임 만들어줘" : 간단한 게임 완성
    - 빠른 프로토타입 제작: MVP
      - 비전문가도 AI로 직접 프로토타입 제작 가능
      - 개발 속도 향상, 기술 장벽 완화

    - **Vibe 코딩 결과물 ?= Production System**
      - AI가 생성한 코드는 복잡한 상황에 취약, 전문 개발자의 코드 검토 및 수정이 필요
      - 아이디어 우선의 함정: 실현 가능성보다 컨셉에 집중될 수 있음
      - 개발자의 영역: Vibe 코딩만으로는 모든 문제 해결 불가
        - » 복잡한 알고리즘, 성능 최적화 등: 숙련된 개발자의 영역

- **챗GPT의 Vibe 코딩 1**
  - 코드 인터프리터
    - 챗GPT에서 바이브코딩
      - 코드 명령어 입력: "데이터를 분석하여 매출 증가율을 시각화하는 파이썬 코드를 작성해줘."
      - 코드 생성: 챗GPT는 명령어를 해석하여 파이썬 코드를 자동으로 생성

python 복사 편집

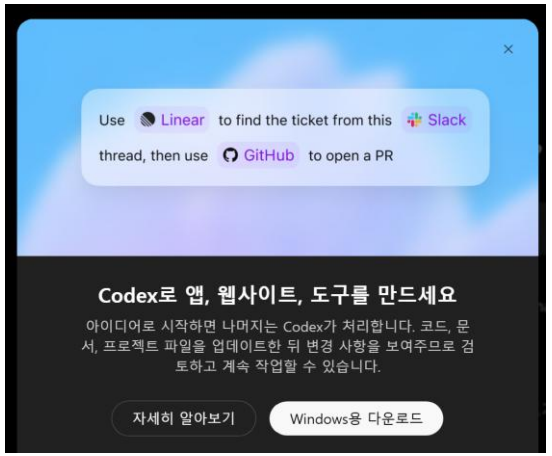
```python
import pandas as pd
import matplotlib.pyplot as plt

# 가상 데이터 생성
data = {'월': ['1월', '2월', '3월', '4월'], '매출': [100, 150, 200, 300]}
df = pd.DataFrame(data)

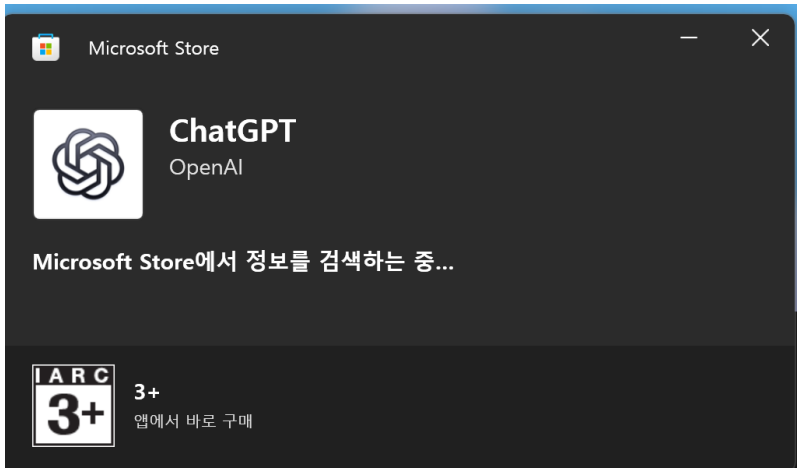
# 시각화
plt.plot(df['월'], df['매출'])
plt.title('매출 증가율')
plt.show()
```

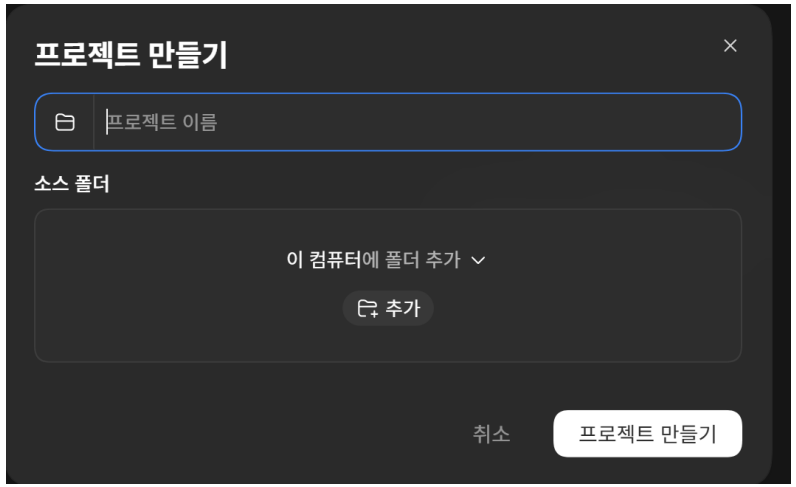
- **챗GPT의 Vibe 코딩 2**
  - 자연어 명령을 코드로 변환하여 실행: "명령 기반 코드 생성"을 실현
  - **Codex**
    - 개발 Agent
    - CLI 버전 또는 IDE Extension으로 사용
    - 로컬 개발환경에서 활용이 필요
    - 여러 파일을 동시에 접근하여 프로젝트 완성

- **챗GPT의 Vibe 코딩 2**
  - Codex: 설치 전 사내 보안 준수여부 확인 / 800MB





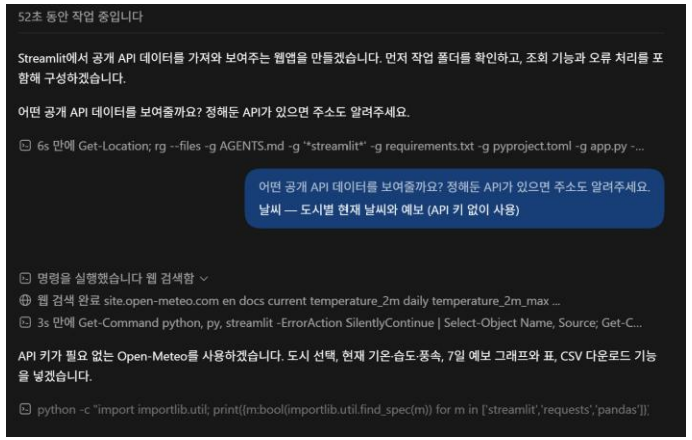


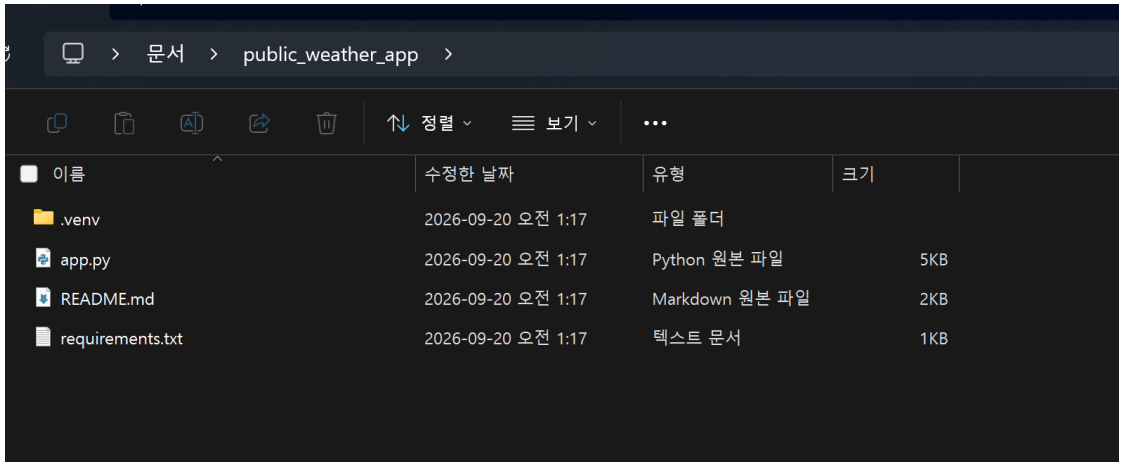


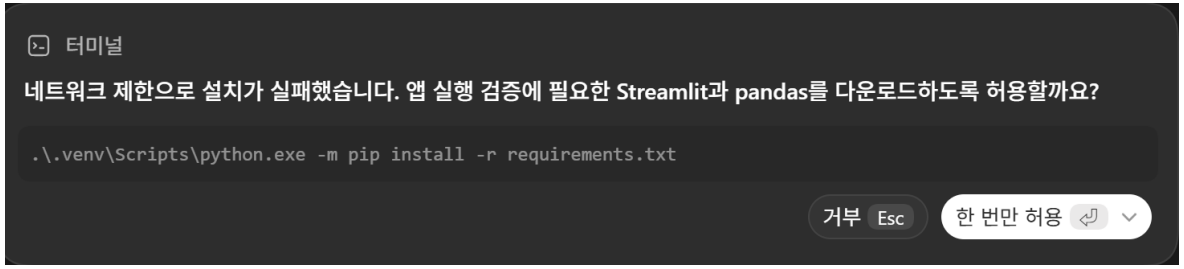
IV. Vibe coding 소개

- **챗GPT의 Vibe 코딩 2**
  - Codex

Streamlit으로 웹앱 제작, 공개된 api에서 값을 가져와서 보여주는 기능

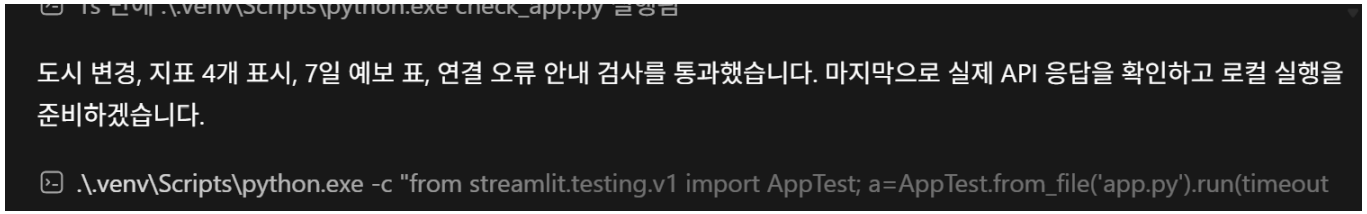


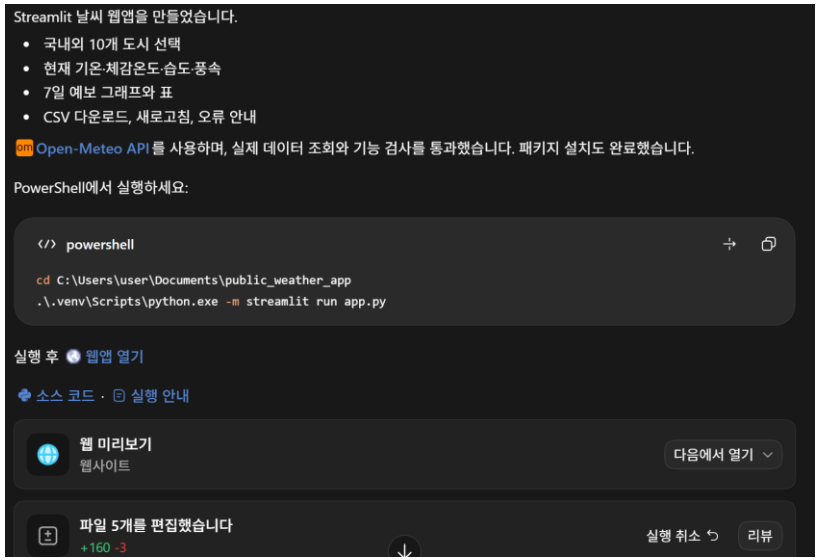


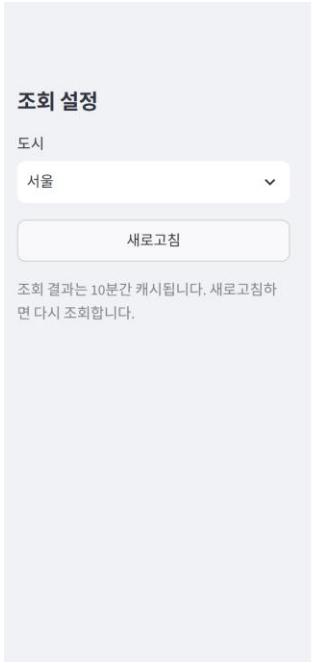


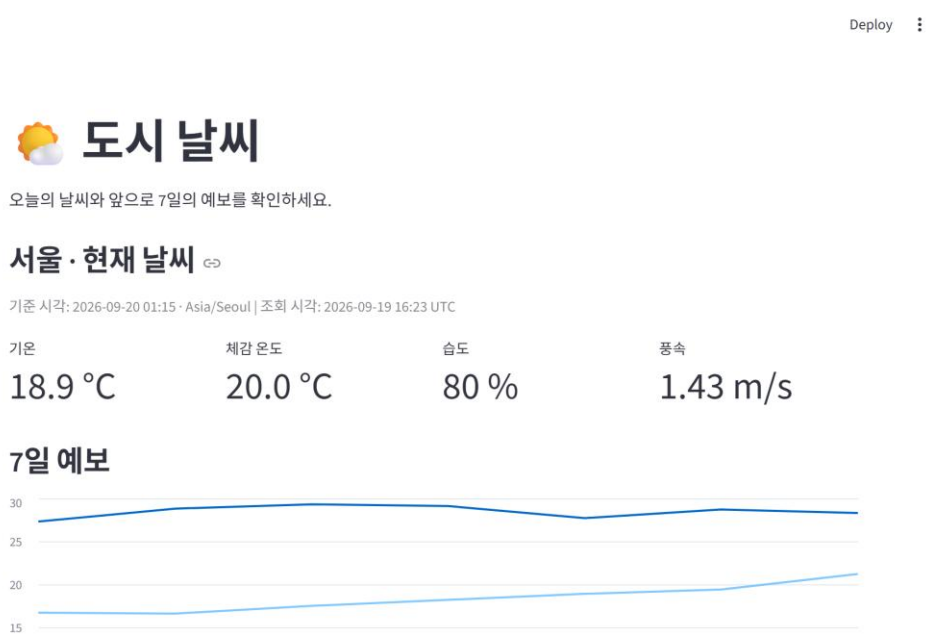
IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 2
  - Codex



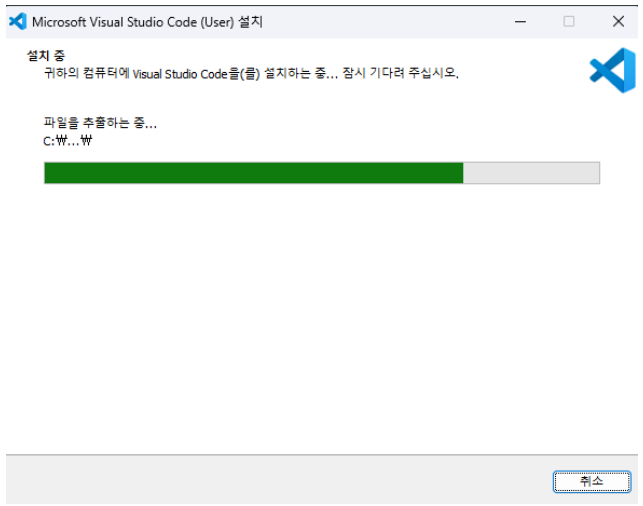


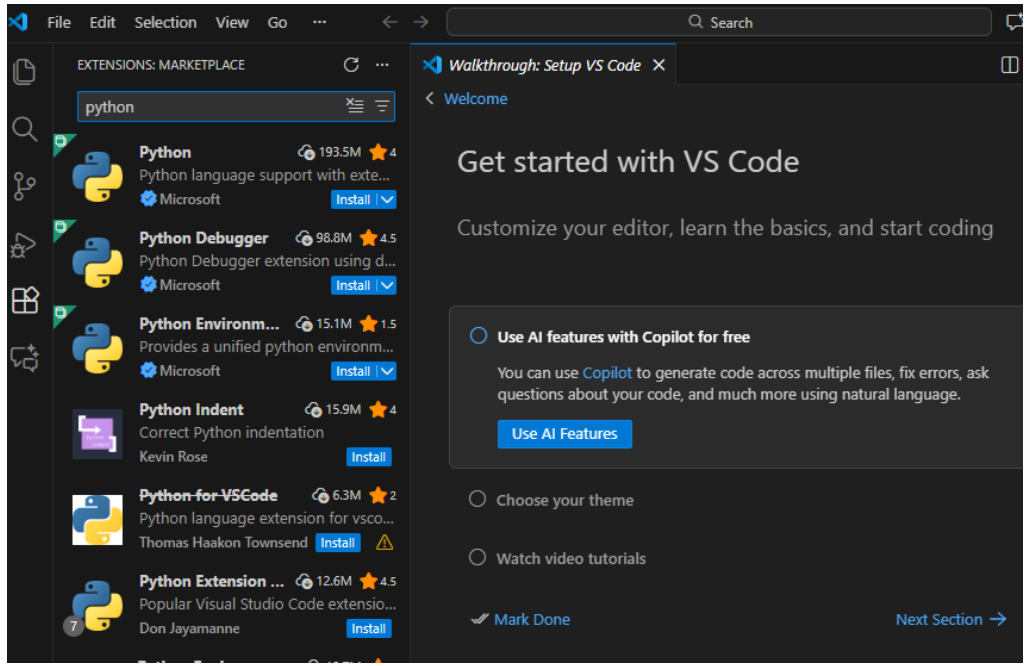




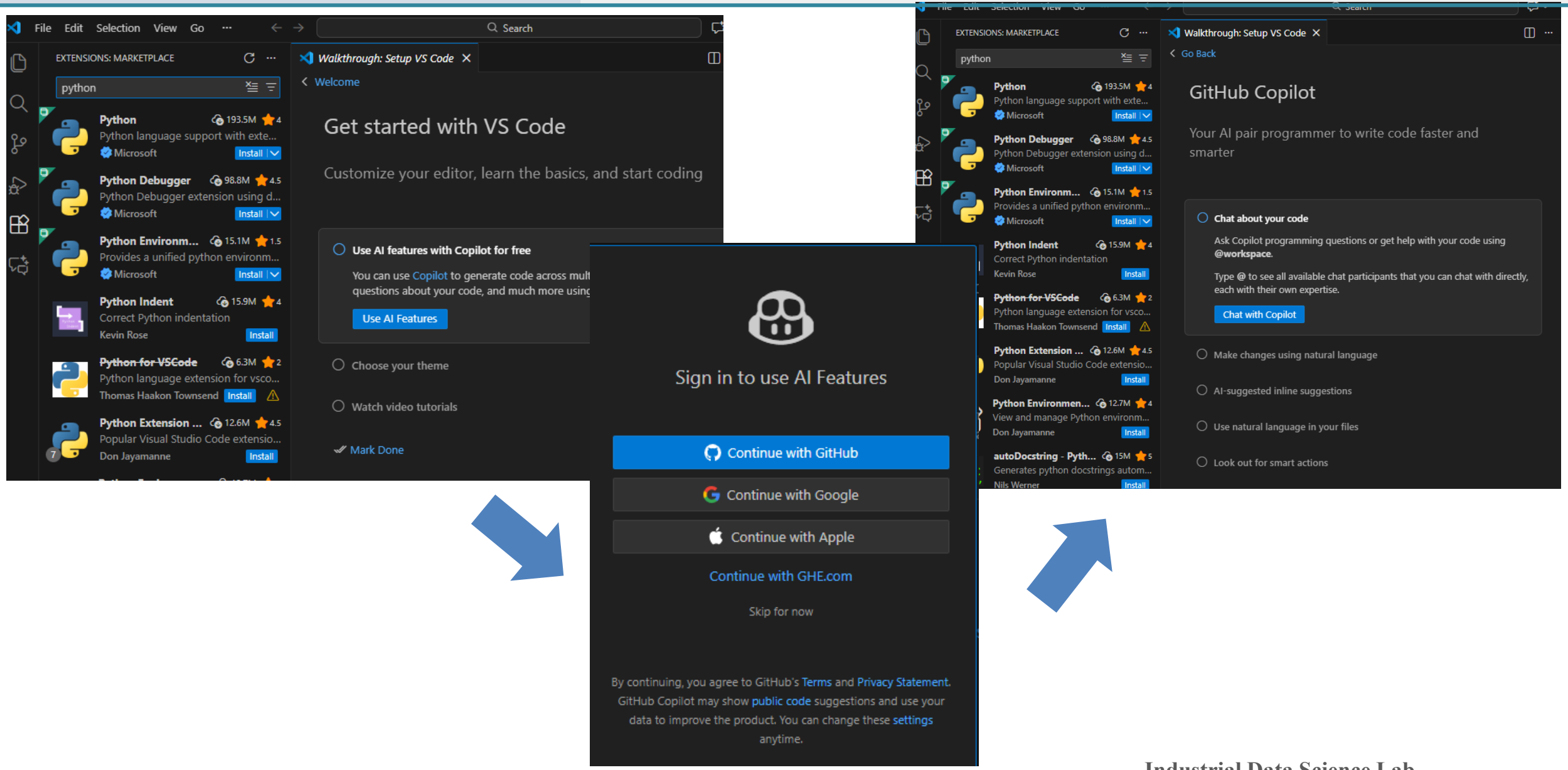
IV. Vibe coding 소개

- **챗GPT의 Vibe 코딩 3: VS Code + LLM**
  - VS Code 설치 -> MS 제공 Python 설치 -> Extension에서 챗gpt 또는 copilot, copilot chat 설치-> Github계정 연동으로 copilot 사용





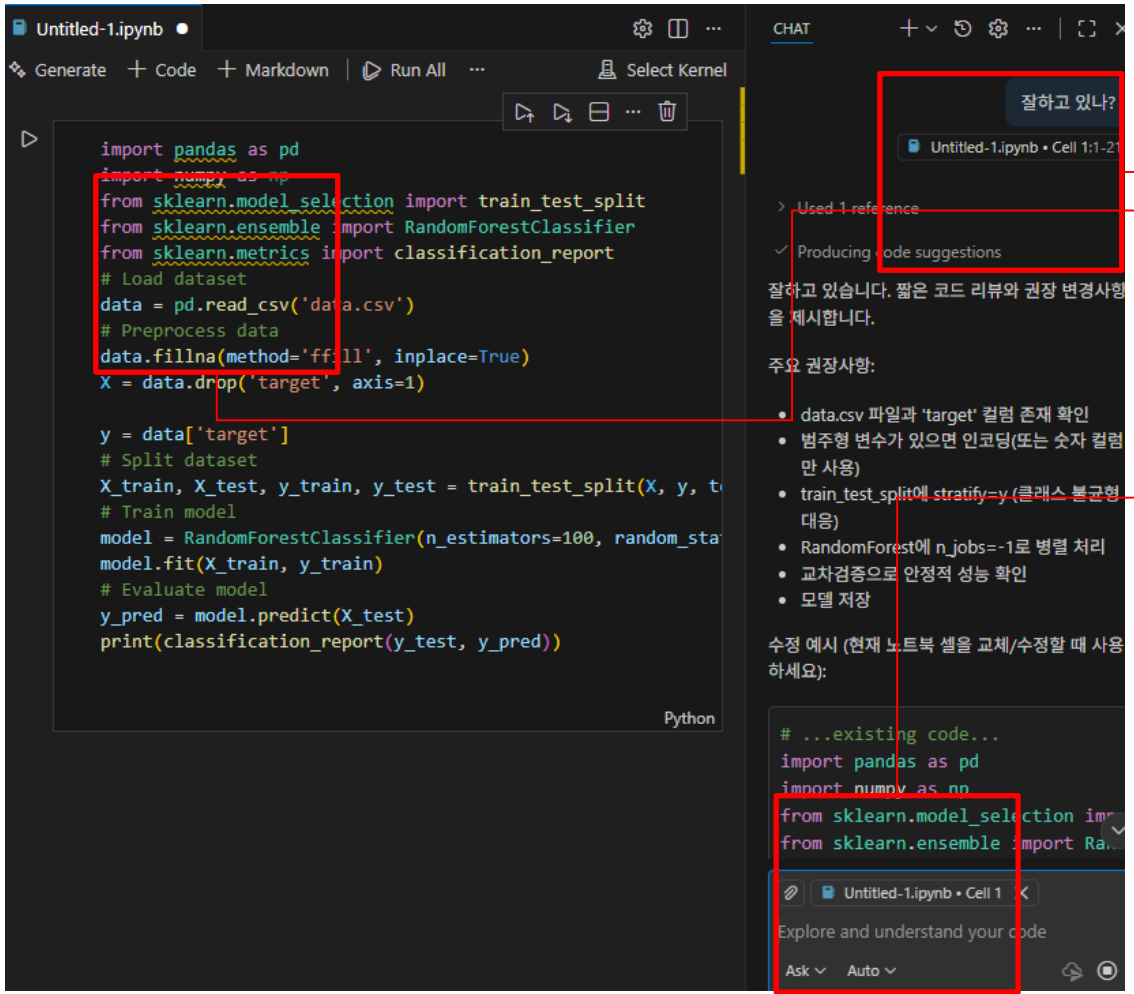
IV. Vibe coding 소개



IV. Vibe coding 소개

- **챗GPT의 Vibe 코딩 3: VS Code + LLM**

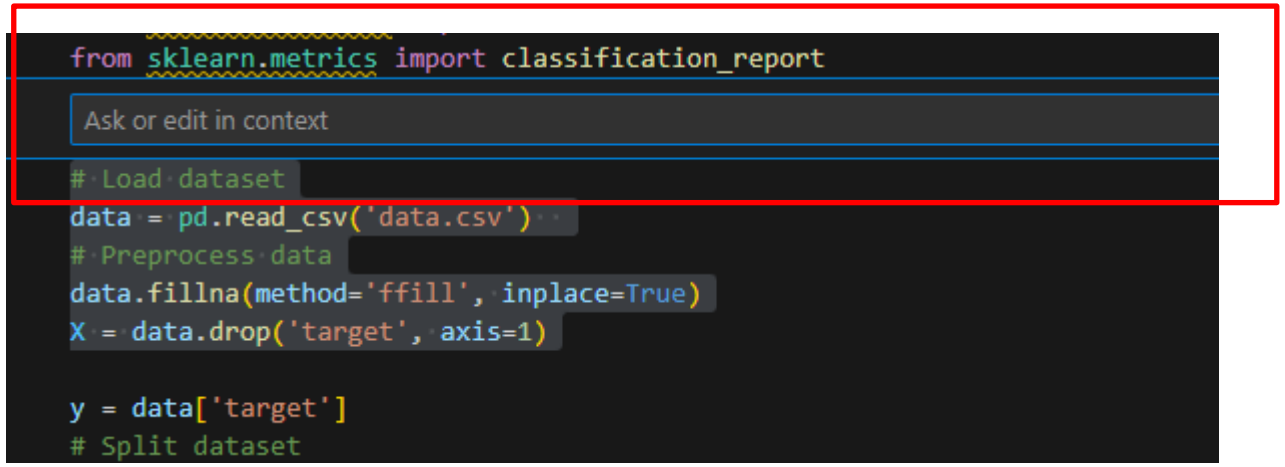
  - New File -> Python 또는 Jupyter Notebook 선택



  - 주석에 한글로 코드에 대한 설명->자동완성
  - 채팅 창에 프롬프트로 코드 요청->이후 반영
  - Ask, Edit, Agent 모드

IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 3: VS Code + LLM



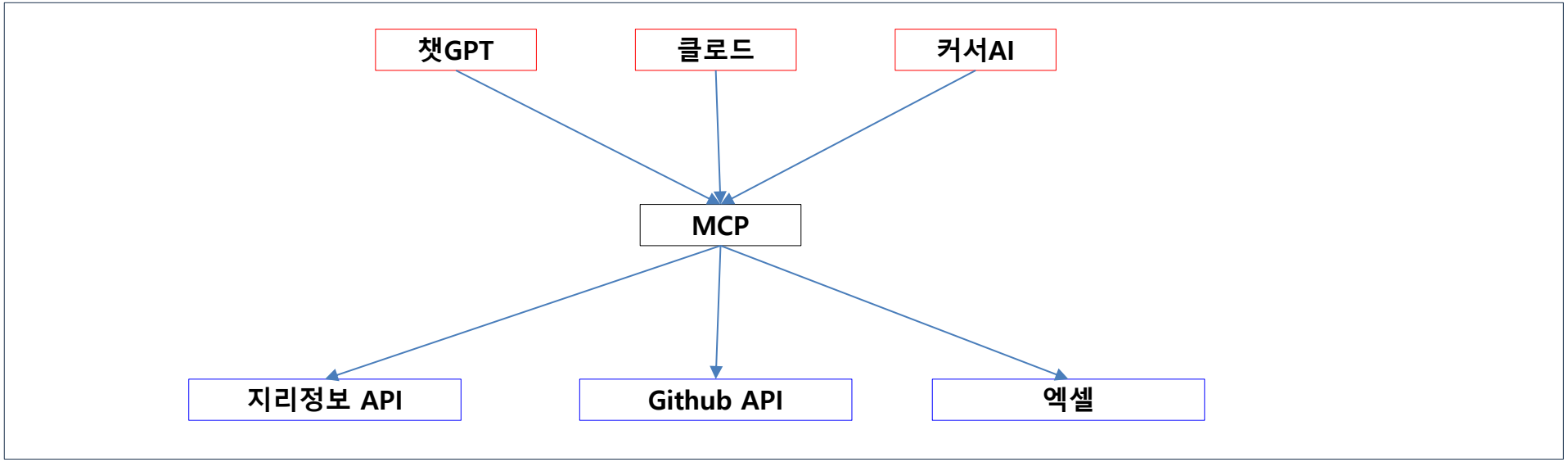
- 영역 지정 후 CTRL+I
- 코드 리팩토링 또는 설명 요청

V. MCP 소개

- **MCP**
  - AI agent: "LLM+도구"로 실제 행동을 수행시키는 기술
  - **Model Context Protocol**: LLM이 외부의 다양한 도구(tool)와 구조화된 방식으로 인터랙션하는 프로토콜
    - 도구(tool): LLM 외부의 데이터/정보를 다룰 수 있는 애플리케이션/API 등 LLM 외부의 기능들
    - '범용 어댑터' : Tool이 MCP를 따르면, 클로드 데스크톱, 커서AI에서도 연동
    - 기존 Open AI의 Function calling, 클로드의 tool_use 메시지구조, Langchain 의 Agent 기반 등 다양한 방식이 존재

V. MCP 소개

- **MCP**
  - 모델과 도구의 연결



V. MCP 소개

- **MCP**
  - MCP 서버: tool, 스미더리 등에서 공용으로 호스팅되어 제공되기도 함.
  - MCP 클라이언트: MCP서버에 요청보내고 결과를 받아 호스트에 전달, 예: 커서 내부LLM 호출 모듈, 클로드 데스크톱의 MCP 요청 처리기 등
  - MCP호스트: 클로드데스크톱, 커서, 윈드서프 등 LLM 기반 App, LLM

  - 도구 정의 및 불러오기
    - 각 도구는 독립된 MCP 서버로 구현: 코드 실행기, 파일분석기, 이메일발송기 등
    - JSON으로 도구 기능 표현
    - 클라이언트는 LLM 으로 JSON을 전달
    - LLM 도구 선택 및 호출

V. MCP 소개

- **주요 MCP 클라이언트**
  - Claude Desktop: 사용자가 직접 MCP 서버 연결 (JSON 도구 목록 인식)
  - 커서 AI: AI기반 코드 편집기, VS Code를 기반으로 GPT 또는 클로드 모델 활용
  - 윈드서프(Windsurf)
    - 워크 플로우 중심의 AI 코드 에이전트
    - 여러 MCP 도구를 조합해 자동화된 작업 흐름 구성
  - Smithery: 여러 MCP 서버들의 집합, 워크플로우 생성

V. MCP 소개

- **MCP 통신 방식**
  - Stdio(표준 입출력)
    - MCP 서버가 터미널에서 실행
    - Stdio로 서버와 클라이언트가 데이터 주고받음
    - 서버와 클라이언트가 동일한 컴퓨터, 빠른 실행
  - SSE(Server Sent Events)
    - HTTP 기반 스트리밍으로 데이터 송수신, 웹 서버 형태의 MCP서버에 적용
    - HTTP Post 방식
    - 다중 클라이언트 및 세션 관리, 실시간 응답 및 스트리밍에 유리

V. MCP 소개

- **MCP 구성 요소**
  - 도구: @mcp.tool()로 등록
  - 리소스: @mcp.resource()로 등록, 참조용 데이터 또는 문맥
  - 프롬프트: @mcp.prompt()로 등록

- **클라이언트에 도구 등록하는 다양한 방법**
  - 모든 플랫폼: 파이썬코드로 구현된 도구를 직접 실행
  - 커서 등: Command 호출 등록
  - 커서: 정적 등록, mcp.json으로 설정하여 클라이언트 자동 등록
  - GUI 등록: 클로드 데스크톱 개발자 모드
  - 원격서버 연결: 웹 환경 전용, 웹 서버 필요

V. MCP 소개

- **MCP 단계별 이해**

① 프롬프트로 질문 또는 명령 입력

② LLM이 입력 분석 후 외부 정보 필요여부 판단

③ 필요 시 MCP 요청 생성: 외부 도구의 호출 준비

④ MCP 서버에서 요청 수신

⑤ MCP 서버가 요청된 도구(기능, 실행 등)를 실행

⑥ 클라이언트로 결과 반환

⑦ LLM이 MCP 결과를 포함해 자연어로 답변 생성

V. MCP 소개

- **챗GPT + MCP :** AI가 파일/DB/GitHub/업무 시스템 등 외부 세계와 연결되는 표준 인터페이스
  - ChatGPT
    - 외부 앱·데이터 소스를 MCP 기반으로 연결 가능
    - 연결된 시스템의 정보 검색·조회 및 일부 작업 수행
  - Codex
    - MCP 서버를 연결하여 개발 환경 및 외부 도구 활용
    - 로컬 개발환경과의 연계에 특히 유용
  - OpenAI API
    - Responses API / Agents 환경에서 MCP 도구 사용 가능
    - 자체 서비스 및 연구 시스템과의 통합 가능

V. MCP 소개

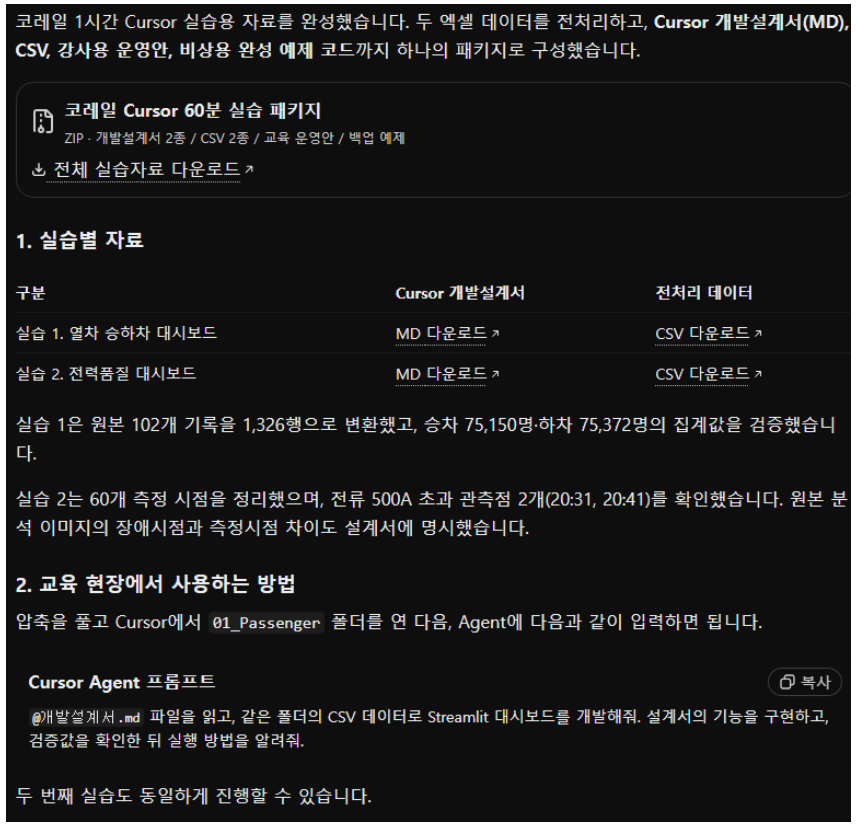
- **챗GPT 플러그인**
  - Plugin = 사용자가 설치·사용하는 기능 패키지
  - App = Google Drive, DB, 사내 시스템 등 외부 서비스 연결
  - MCP = AI와 외부 App/도구를 연결하는 표준 프로토콜
  - 흐름: ChatGPT → Plugin → App → MCP → 외부 시스템
    - 모든 플러그인이 MCP를 쓰지는 않음
    - MCP 기반 App이 플러그인에 포함될 수 있음

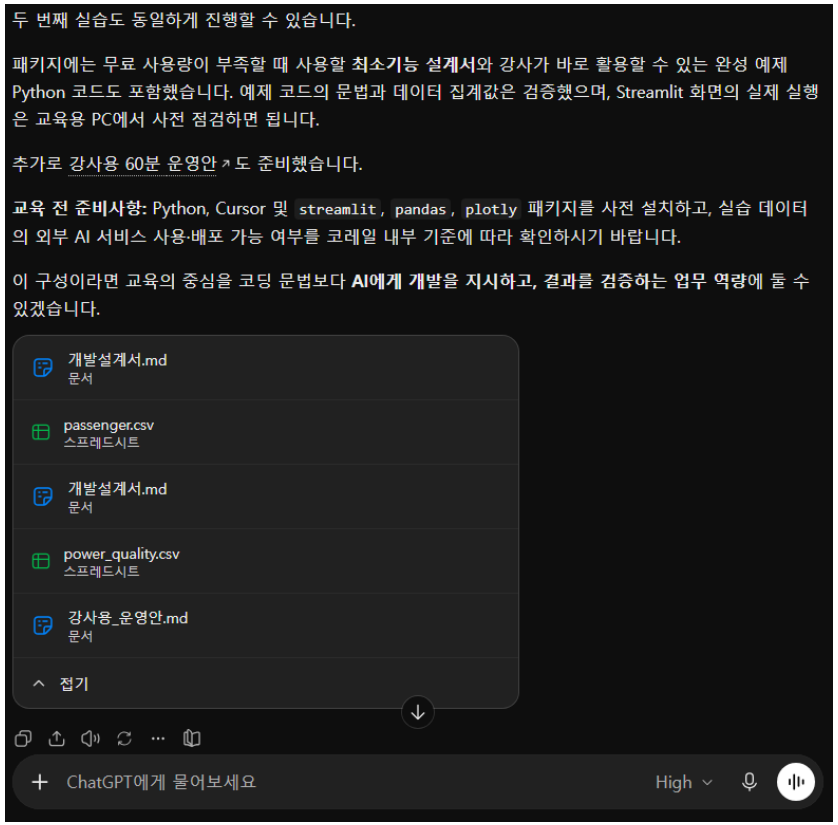
VI. LLM 실습

- **환경:**
  - 커서 / Python 3.13 또는 이상: ***파이썬설치.pdf*** 참고
- **참고:**
  - Markdown:텍스트에 간단한 기호를 사용하여 제목, 목록, 표, 코드 등을 구조화하는 경량 마크업 언어. 작성이 쉽고 가독성이 높으며, AI가 문서 구조를 이해하기 좋음. AI 작업 지시서 등에 사용
  - Codex: OpenAI의 AI 코딩 에이전트. 자연어 명령으로 코드를 작성/수정/실행하며, 터미널/IDE/웹 등에서 활용 가능
  - Claude Code: Anthropic의 AI 코딩 에이전트. 프로젝트 전체 코드를 이해하고 개발, 오류 수정, 테스트 등을 자동 수행. 터미널과 IDE에서 사용 가능.
  - Cursor: AI 기반 코딩 도구. 자연어로 개발을 지시하면 AI가 코드를 생성/수정하고 실행을 지원. 다양한 AI 모델 활용 가능.
  - Cursor IDE: VS Code 기반의 AI 통합 개발환경(IDE). 코드 편집기, 파일 탐색기, 터미널, AI Agent 등을 하나의 화면에서 제공
  - Cursor IDE: 사람이 코드를 확인·편집하면서 AI와 협업하는 통합 개발환경 중심

VI. LLM 실습

- **LLM으로 개발 설계**
  - 예: 챗gpt로 개발 설계 후 MD 파일로 저장
  - README_시작하기.MD 확인





- **실습1**
  - 압축파일 전체를 압축 해제하고, Cursor에서 01_Passenger 폴더 하나만 먼저 열어 시작
    - Cursor의 **File → Open Folder**로 `01_Passenger` 선택.
    - Agent 대화에 아래 문구 입력 (파일 첨부 메뉴 또는 `@` 입력 후 해당 파일을 선택해도 됨):
      - **@개발설계서.md 파일을 먼저 읽어줘. 같은 폴더의 passenger.csv로 app.py와**
      - **requirements.txt를 작성해줘. 요구사항 중 핵심 기능부터 개발하고,**
      - **검증값(전체 합계)이 일치하는지 확인한 후 실행 방법을 알려줘.**

VI. LLM 실습

- 실습1

@개발설계서.md 파일을 먼저 읽어줘. 같은 폴더의 passenger.csv로 app.py와 requirements.txt를 작성해줘. 요구사항 중 핵심 기능부터 개발하고, 검증값(전체 합계)이 일치하는지 확인한 후 실행 방법을 알려줘.

Thought briefly

설계서와 데이터 파일을 먼저 확인한 뒤, 핵심 기능부터 구현하겠습니다.

Explored 개발설계서.md, 1 search

설계서를 확인했습니다. CSV 구조와 합계를 검증한 뒤 `app.py`와 `requirements.txt`를 작성합니다.

Explored 2 files

핵심 기능부터 `app.py`와 `requirements.txt`를 작성한 뒤, CSV 합계를 검증하겠습니다.

Editing 2 files, ran 2 commands +199

Verify CSV totals without sandbox python

```
$ python -c "import pandas as pd; df=pd.read_csv('passenger.csv', encoding='utf-8-sig', dtype=
{'train_no': str, 'month': str}); print('rows', len(df)); print('pairs',
df[['month','train_no']].drop_duplicates().shape[0]); print('months',
sorted(df['month'].unique().tolist())); print('boardings', int(df['boardings'].sum()));
print('alightings', int(df['alightings'].sum())); print('traffic',
int(df['boardings'].sum()+df['alightings'].sum())); print('sample_train_no',
df['train_no'].head(3).tolist())"
```

Allowlist Skip Always Run Run

생성한 파일: `app.py`, `requirements.txt`
실행 명령: `python -m pip install -r requirements.txt` 후 `python -m streamlit run app.py`
검증 통과 여부: 통과
남은 오류: 없음

기본 선택(전체) 합계는 설계서와 같습니다. 승차 **75,150**, 하차 **75,372**, 승하차 **150,522**. 행 수 1,326, 월별 열차 조합 102개, 월 `2026-01`~`2026-06`, 열차번호 `02502`처럼 앞의 0도 유지됩니다. 월 필터에서 `2026-01`을 빼면 KPI가 **56,920 / 57,913 / 114,833**으로 바뀌는 것도 확인했습니다.

같은 폴더에서 위 명령으로 실행하면 브라우저에 대시보드가 열립니다. 지금 로컬에서는 `http://localhost:8501`로 이미 떠 있습니다.

Just now

2 Files Changed Review

requirements.txt +3

app.py +196

검증값은 일치합니다. 패키지를 설치한 뒤 앱을 띄워 화면을 확인하겠습니다.

Pending approval

Install dashboard Python dependencies python

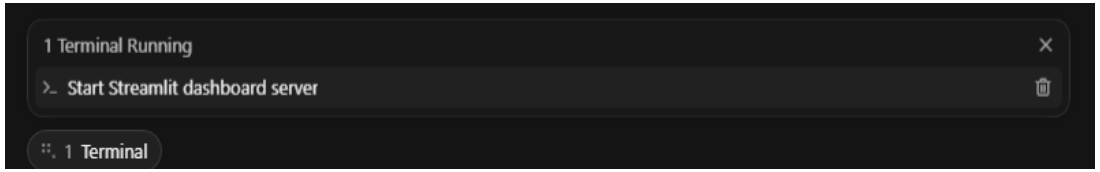
```
$ python -m pip install -r requirements.txt
```

Allowlist Skip Always Run Run

VI. LLM 실습

- 실습1


http://localhost:8501
Deploy
필터
기본값은 전체입니다. 선택을 모두 지우면 0건으로 안내합니다.
운행년월
2026-01 × 2026-02 × 2026-03 × 2026-04 × 2026-05 × 2026-06 ×
열차종
ITX-마음 × 누리로 × 무궁화호 × 새마을호 ×
노선그룹
1영동영주강릉 × 중앙동대구-강릉 ×
정차역
강릉 × 도계 × 동백산 × 동해 × 묵호 × 신기 × 정동진 × 철암 ×
코레일 열차별 승하차 현황 대시보드
실습용 분석 화면 / 자료의 활용 권한 확인
승차인원 합계
75,150 명(집계)
하차인원 합계
75,372 명(집계)
승하차 건수 합계
150,522 건
월별 추이
구분
승차
하차
15k
10k
5k
0
인원(집계)
Jan 2026
Feb 2026
Mar 2026
Apr 2026
May 2026
Jun 2026
운행년월


1 Terminal Running
Start Streamlit dashboard server
1 Terminal

- **실습2**
  - Cursor에서 02_PowerQuality 폴더 열어 시작
    - Cursor에서 `02_PowerQuality` 폴더 열기.
    - Agent에 아래 문구 입력:
      - @개발설계서.md 내용을 읽어줘. power_quality.csv를 이용하여
      - 전차선 전류·전압 시계열을 시각화하고 500A를 초과하는 지점을
      - 붉게 표시하는 Streamlit app.py를 작성해줘.

VI. LLM 실습

- 실습2

@개발설계서.md 내용을 읽어줘. power_quality.csv를 이용하여
전차선 전류·전압 시계열을 시각화하고 500A를 초과하는 지점을
붉게 표시하는 Streamlit app.py를 작성해줘.

Thought briefly

설계서와 데이터 파일을 먼저 확인한 뒤, 그에 맞춰 Streamlit 앱을 작성하겠습니다.

Explored 2 files, 5 searches

설계서 요구사항을 반영해 `app.py`와 `requirements.txt`를 작성합니다.

Editing 2 files, explored 1 search, ran 3 commands +200

Validate CSV stats against design spec python

```
$ python -c "import pandas as pd; df=pd.read_csv('power_quality.csv');
df['timestamp']=pd.to_datetime(df['timestamp']); df['current_a']=pd.to_numeric(df['current_a']);
print(len(df), df['timestamp'].min(), df['timestamp'].max()); print((df['current_a']>500).sum());
print(df.loc[df['current_a']>500, ['timestamp','current_a']].to_string());
print(df['current_a'].max())"
```

Allowlist Skip Always Run Run

Start Streamlit dashboard server python

```
$ python -m streamlit run app.py --server.headless true
```

Allowlist Skip Always Run Run

검증 결과

CSV 60행, 구간 `2026-06-25 20:00:28` ~ `20:59:28`이 맞습니다.

기본 임계치 500 A에서:

- 초과 2건
- `20:31:28` = 1,869.529 A
- `20:41:28` = 10,903.695 A (전체 최대)

임계치를 20,000 A로 올리면 초과 수는 0이 되고, "선택한 임계치를 초과하는 관측점이 없습니다." 문구로 바뀌는 것도 확인했습니다.

판정 한계

이 앱은 **측정값 탐색용**입니다. `current_a > 임계치`만 표시하며, 장애 시각·원인과 자동으로 연결하지 않습니다. 첨부 분석 자료의 20:29경 서술과 CSV 피크(20:31, 20:41)는 다를 수 있고, 전압·THD 극단값도 원자료 그대로 보여 줍니다.

Stopped Start Streamlit dashboard server

Just now

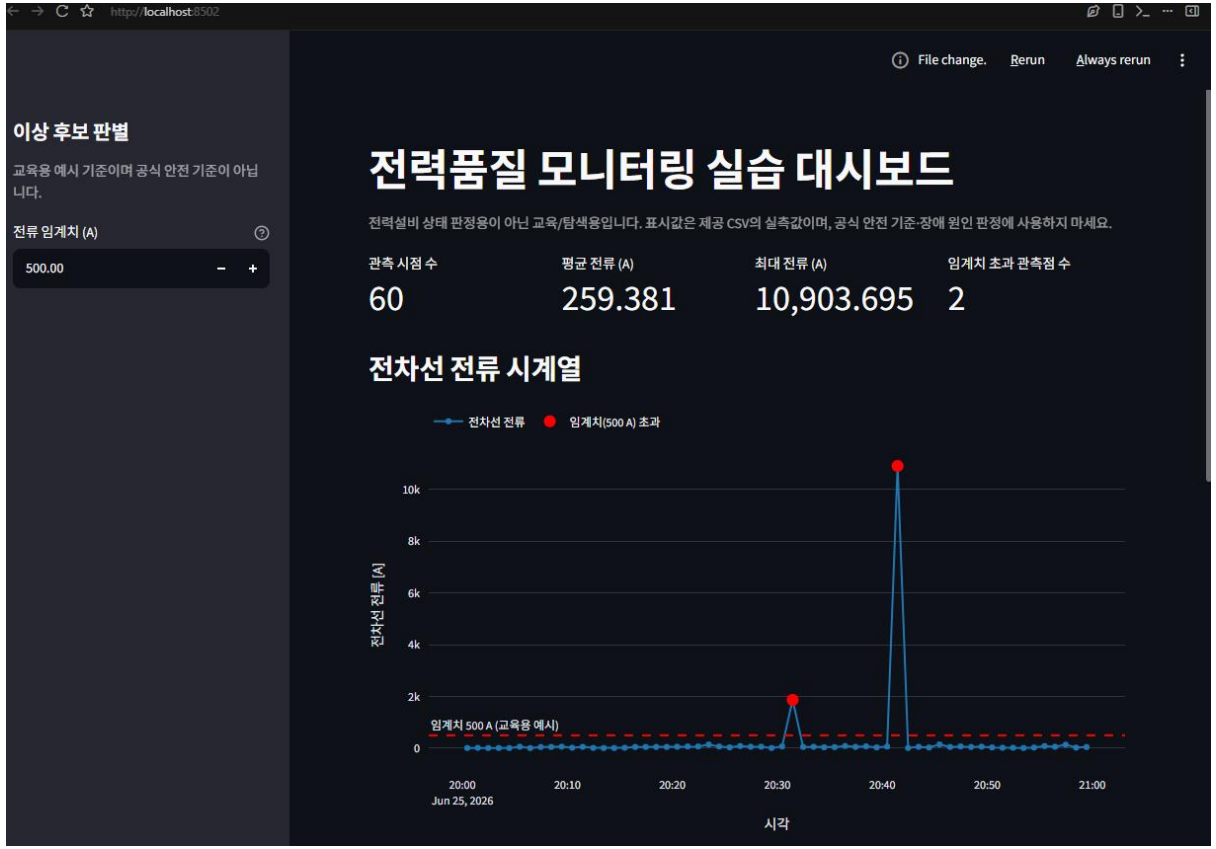
2 Files Changed Review

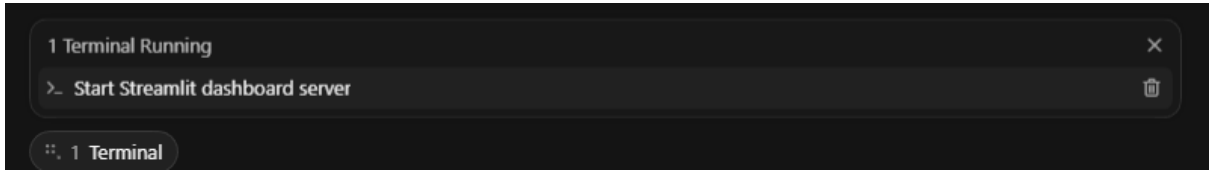
requirements.txt +3

app.py +197

VI. LLM 실습

- 실습2


http://localhost:8502
File change.
Rerun
Always rerun
이상 후보 판별
교육용 예시 기준이며 공식 안전 기준이 아닙니다.
전류 임계치 (A)
500.00
전력품질 모니터링 실습 대시보드
전력설비 상태 판정용이 아닌 교육/탐색용입니다. 표시값은 제공 CSV의 실측값이며, 공식 안전 기준·장애 원인 판정에 사용하지 마세요.
관측 시점 수
60
평균 전류 (A)
259.381
최대 전류 (A)
10,903.695
임계치 초과 관측점 수
2
전차선 전류 시계열
전차선 전류
임계치(500 A) 초과
10k
8k
6k
4k
2k
0
전차선 전류 [A]
임계치 500 A (교육용 예시)
20:00
Jun 25, 2026
20:10
20:20
20:30
20:40
20:50
21:00
시각


1 Terminal Running
Start Streamlit dashboard server
1 Terminal

Industrial Data Science Lab

Contact:
won.sang.l@kangwon.ac.kr
https://sites.google.com/view/idslab